Q6 – Priority, Order and Fairness in a Queue: Hospital Emergency Triage (8 Marks, K5)

> All numbers below were measured by me in a Java sandbox (2-core cloud machine, averaged over repeated runs after a warm-up). Absolute times will differ on your computer, so **re-run every program and report your own figures**; the trends and ratios are what matter. Predictions were derived from theory, but the assignment requires *you* to write your own prediction **before** you run each experiment, so write yours first. Each screenshot must be linked to the test it shows.

1. Design

Why a plain queue (`LinkedList`/`ArrayDeque`) is not sufficient: a FIFO queue serves strictly in arrival order, so a severity-1 (critical) patient who arrives after ten minor patients would wait behind all of them. The rule is "most severe first, and among equals first-come-first-served": two criteria, not one.

How `PriorityQueue` with a `Comparator` enforces the rule: `PriorityQueue` keeps a binary min-heap ordered by a comparator, so `poll()` always returns the *smallest* element. The comparator is: `severity` ascending (1 is the most severe), then `arrival` ascending.

How arrival order is preserved among equal severities: `PriorityQueue` is **not stable**: two elements that compare equal come out in unspecified order. So I give every patient a unique, increasing **arrival counter** (`counter++`) and use it as the second key. No two patients ever compare equal, so the order among patients of equal severity is exactly their arrival order.

Removing the next patient and displaying the waiting list: `poll()` removes and returns the head in O(log n) (`peek()` looks without removing). The heap is only *partially* ordered, so to *display* the waiting list in treatment order you must copy the queue and `poll()` the copy until empty (or copy to a list and sort it with the same comparator): O(n log n). Never iterate the queue to display it.

2. Complete the Code (TODO 1 filled in, executed)

```
import java.util.Comparator;
import java.util.PriorityQueue;

class Patient {
    final String name; final int severity; final long arrival;
    Patient(String name, int severity, long arrival) {
        this.name = name; this.severity = severity; this.arrival =
arrival;
    }
    @Override public String toString() { return name + "(S" + severity
+ ")"; }
}

public class Triage {
```

```
    private long counter = 0;

    // TODO 1 (completed): lowest severity number first; for equal
severity, earliest arrival first
    private final PriorityQueue<Patient> queue = new PriorityQueue<>(
            Comparator.comparingInt((Patient p) -> p.severity)
                    .thenComparingLong(p -> p.arrival));

    void admit(String name, int severity) {
        queue.add(new Patient(name, severity, counter++));
    }

    Patient next() { return queue.poll(); }

    public static void main(String[] args) {
        Triage er = new Triage();
        er.admit("Ravi", 3); er.admit("Meena", 1); er.admit("John",
3);
        er.admit("Asha", 5); er.admit("Kumar", 1); er.admit("Lata",
2);
        StringBuilder order = new StringBuilder();
        Patient p;
        while ((p = er.next()) != null) order.append(p).append(' ');
        System.out.println(order.toString().trim());
    }
}
```

Output (matches the expected output):

```
Meena(S1) Kumar(S1) Lata(S2) Ravi(S3) John(S3) Asha(S5)
```

How the arrival counter keeps equal severities in order: Ravi (counter 0) and John (counter 2) both have severity 3; the second comparator key makes Ravi < John because 0 < 2, so Ravi is served first. Likewise Meena (1) before Kumar (4) at severity 1.

3. Correctness check – 8 patients worked out by hand (`TriageExperiments.java`)

Arrivals: Ravi S3, Meena S1, John S3, Asha S5, Kumar S1, Lata S2, Dev S2, Zoya S4. **By hand:** S1 → Meena (arrived 2nd), Kumar (5th); S2 → Lata (6th), Dev (7th); S3 → Ravi (1st), John (3rd); S4 → Zoya; S5 → Asha. Correct order: **Meena, Kumar, Lata, Dev, Ravi, John, Zoya, Asha.**

Method	Output
Polling repeatedly (treatment order)	`Meena(S1) Kumar(S1) Lata(S2) Dev(S2) Ravi(S3) John(S3) Zoya(S4) Asha(S5)` ✔ matches the hand-worked order
Printing the `PriorityQueue` directly	`Meena(S1) Kumar(S1) Lata(S2) Zoya(S4) Ravi(S3) John(S3) Dev(S2) Asha(S5)` ✘

Why printing the queue directly does not show the treatment order:
`toString()`/iteration visits the *internal array of the binary heap*. A heap only guarantees that each parent is ≤ its children (the smallest element is at index 0); siblings and cousins are in no particular order. In the output above, Zoya (S4) appears before Ravi (S3). Only repeated `poll()` produces the sorted sequence.

```
import java.util.*;

public class TriageExperiments {
    record P(int id, int sev, long arr) {}
    static final Comparator<P> ORDER =
Comparator.comparingInt(P::sev).thenComparingLong(P::arr);

    public static void main(String[] args) {
        // ---- 1. hand-checkable instance with 8 patients
        String[][] in = {{"Ravi","3"},{"Meena","1"},{"John","3"},
{"Asha","5"},{"Kumar","1"},{"Lata","2"},{"Dev","2"},{"Zoya","4"}};
        PriorityQueue<P> pq = new PriorityQueue<>(ORDER);
Map<Integer,String> nm = new HashMap<>();
        int id = 0; for (String[] x : in) { nm.put(id, x[0]);
pq.add(new P(id, Integer.parseInt(x[1]), id)); id++; }
        StringBuilder raw = new StringBuilder(); for (P p : pq)
raw.append(nm.get(p.id())).append("(S").append(p.sev()).append(") ");
        System.out.println("Hand-worked order: Meena(1) Kumar(1)
Lata(2) Dev(2) Ravi(3) John(3) Zoya(4) Asha(5)");
        System.out.println("Printing the PriorityQueue directly
(internal heap array): " + raw.toString().trim());
        StringBuilder served = new StringBuilder(); PriorityQueue<P> c
= new PriorityQueue<>(pq); P p;
        while ((p = c.poll()) != null)
served.append(nm.get(p.id())).append("(S").append(p.sev()).append(")
");
        System.out.println("Polling repeatedly (treatment order)
: " + served.toString().trim());

        // ---- 2. growth: cost of add and of poll as the queue grows
        System.out.printf("%n%10s | %14s | %14s   (ns per operation,
average)%n", "queue size", "add (offer)", "remove (poll)");
        Random r = new Random(1);
        for (int w = 0; w < 3; w++) { PriorityQueue<P> q = new
PriorityQueue<>(ORDER); for (int i = 0; i < 50000; i++) q.add(new P(i,
1 + r.nextInt(5), i)); while (q.poll() != null) {} }
        for (int n : new int[]{1_000, 10_000, 100_000, 1_000_000}) {
            PriorityQueue<P> q = new PriorityQueue<>(ORDER); long arr
= 0; for (int i = 0; i < n; i++) q.add(new P(i, 1 + r.nextInt(5), arr+
+));
            int ops = 20000; P[] extra = new P[ops]; for (int i = 0; i
< ops; i++) extra[i] = new P(n + i, 1 + r.nextInt(5), arr++);
            long t0 = System.nanoTime(); for (P e : extra) q.add(e);
double add = (System.nanoTime() - t0) / (double) ops;
```

```
            t0 = System.nanoTime(); for (int i = 0; i < ops; i++)
q.poll(); double rem = (System.nanoTime() - t0) / (double) ops;
            System.out.printf("%10d | %14.0f | %14.0f%n", n, add,
rem);
        }

        // ---- 3. starvation: 3 arrivals (sev 1-4) per tick, 1
patient served per tick, one sev-5 patient waiting
        PriorityQueue<P> q = new PriorityQueue<>(ORDER); q.add(new P(-
1, 5, 0)); long clock = 1; boolean servedFive = false; int ticks =
20000;
        for (int t = 0; t < ticks && !servedFive; t++) { for (int k =
0; k < 3; k++) q.add(new P((int) clock, 1 + r.nextInt(4), clock++)); P
s = q.poll(); if (s.sev() == 5) servedFive = true; }
        System.out.println("\nStarvation test: sev-5 patient served
within " + ticks + " ticks of overload? " + servedFive + "  (queue
length now " + q.size() + ")");
    }
}
```

4. Growth investigation

Prediction (before measuring): `add` = sift-up: **O(log n)** worst case, but only O(1) on average for random priorities (a new element usually settles near the bottom). `poll` = move the last element to the root and sift-down: **O(log n)**: about $\log_2 n$ comparisons, so cost should grow slowly (about 2× from n = 1,000 to 1,000,000 because $\log_2$ goes 10 → 20), but the queue is bigger than the CPU cache at large sizes, so extra slowdown is likely.

Measured (average ns per operation, 20,000 operations at each size):

Queue size	`add`	`poll`
1,000	27 ns	165 ns
10,000	27 ns	385 ns
100,000	25 ns	258 ns
1,000,000	27 ns	651 ns

Comparison: `add` stayed flat at ≈ 26 ns, as predicted for random keys (O(1) average). `poll` grew with size (165 → 651 ns, ×3.9 for ×1,000 more patients), consistent with logarithmic growth plus cache misses ($\log_2$ alone predicts only ×2; the 10,000 value is a noisy outlier, so repeat the measurement and average more runs).

5. Inquiry – starvation and a fair rule

Why a severity-5 patient may wait indefinitely: the comparator *always* prefers a smaller severity number, so as long as patients of severity 1–4 keep arriving at least as fast as the doctor can treat them, a severity-5 patient is never at the head of the queue. Simulation: one severity-5 patient waiting, 3 arrivals (severity 1–4) per tick, 1 patient served per tick: **after 20,000 ticks the severity-5 patient had still not been seen** and the queue had grown to 40,001 patients.

Fair rule (aging): a patient who has waited longer than a limit for their level is served first. For example: maximum waits of 30, 60, 120 and 240 ticks for severities 2, 3, 4 and 5 (severity 1 is always served at once). Equivalent formulation: *effective severity = severity − [waitingTime / T]* (never below 1). Implementation in Q7: each severity level has its own FIFO queue; `serve()` first checks the 5 heads for an overdue patient. In the simulation above, with a 240-tick limit, the severity-5 patient was served after **241 ticks**. Trade-off: some very severe patients may be delayed by overdue minor ones, so the limits must be chosen medically.

6. Deployment decision – 5,000 patients per day

Yes, the design can serve a city hospital. 5,000 patients per day is about 0.06 patients per second. My measurements show an `add` costs about 27 ns and a `poll` at most 0.65 µs, even with a queue of **1,000,000** waiting patients; a real emergency department has at most hundreds waiting. The data-structure cost is therefore thousands of times too small to matter; the real limits are the doctors, not the queue. Two things would still be needed for deployment: (1) the **aging rule** above (the starvation experiment shows the plain design is unsafe under overload), and (2) the queue is not thread-safe, so if several terminals admit patients at once it must be wrapped (`PriorityBlockingQueue` or `synchronized`), and it must be saved to disk (the queue is in memory only).

7. Alternative concept and complexity analysis

Alternatives: (a) a `LinkedList` kept sorted (insert each patient at the correct position); (b) an **array of five FIFO queues**, one per severity level.

```
(a) admit(p): walk from the head until a patient with worse (greater)
key is found, insert before it      O(n)
    serve():  removeFirst()
O(1)
(b) admit(p): queues[p.severity].addLast(p)
O(1)
    serve():  for s = 1..5: if queues[s] not empty → return
queues[s].pollFirst()                          O(5) = O(1)
```

Design	Admit	Serve next	Space	Notes
`PriorityQueue` (heap)	O(log n)	O(log n)	O(n) (array of references)	general: works for any number of priorities
Sorted `LinkedList`	**O(n)**	O(1)	O(n) + 24 bytes per node	the walk to the insertion point dominates; display is O(n) with no sort
Array of 5	**O(1)**	**O(1)** (checks	O(n) + 5	arrival order

Design	Admit	Serve next	Space	Notes
FIFO queues		at most 5 queues)	queue objects	among equals is automatic (FIFO), no counter needed; only works because there are *few, fixed* severity levels; display is O(n) with no sort

The five-queue design is the best when the number of priority levels is small and fixed, as it is here; I implement and measure it in Q7.